

APUNTES - 3 DE SEPTIEMBRE



Compiladores e Intérpretes





Estructura básica de un compilador



01 - Entrada y Salida (p3)



05 - Análisis Semántico (p18)



02 - Estructura Detallada (p7)



06 - Optimizador de Fuente (p22)



03 - Análisis Léxico/Scanner (p11)



07 - Generador de Código (p25)



04 - Análisis Sintáctico/Parser (p19)



08 - Optimizador de Código (p28)



1

ENTRADA Y SALIDA

Entrada y salida

Todo compilador tiene de entrada un lenguaje fuente y de salida un lenguaje objeto. En el proceso, se genera una representación intermedia; que nos ayuda a entender la separación entre el Frontend y el backend de un compilador.



Análisis y síntesis

Analizar es descomponer algo en partes para entenderlo, sintetizar es construir algo de varias piezas sueltas. Un compilador, tiene un frontend que “analiza” y genera esta representación intermedia, y un backend que recibe la representación intermedia y “sintetiza”



Representación intermedia



2

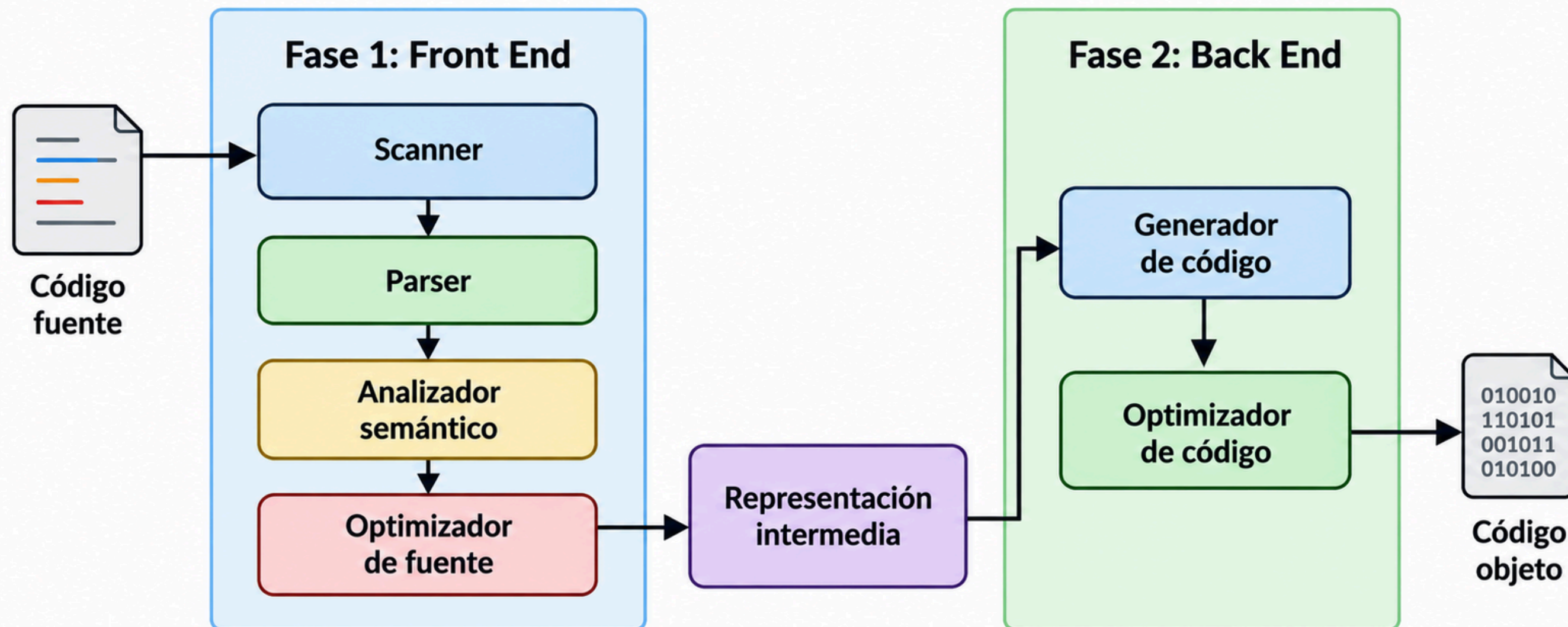
ESTRUCTURA DETALLADA

Pasadas:

Un concepto que ya hemos visto y que es vital que manejemos es el de **pasadas**. Una pasada es una lectura completa del código fuente de inicio a fin. Un compilador puede tener N pasadas, que traen un dilema de que entre más pasadas hay más flexibilidad pero más tiempo de compilación



Fases de compilación



Estructura detallada

Es importante detallar que aunque se vea hermoso y sencillo; el profe advirtió que nos iba a mentir en esta clase



3

ANÁLISIS LÉXICO

(SCANNER)

Análisis léxico

- El léxico se refiere al vocabulario. El Análisis léxico, conocido como scanner, identifica el vocabulario del código fuente según el lenguaje que se espera
- Toma el código fuente de entrada y lo descompone en unidades léxicas mínimas. Generan tokens para el parser.
- Implementado con Autómatas de Estados Finitos.
- Hay herramientas que generan la mayor parte de un scanner.
- Subraya código y lo separa en partes: palabras reservadas, separadores, términos definidos por el programador, operadores, constantes...



Scanner



Entrada





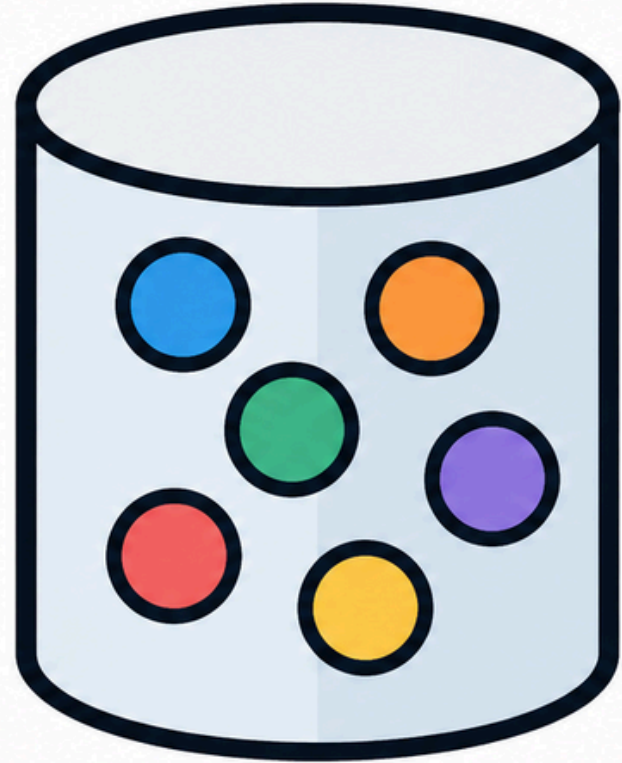
Código fuente





Salida



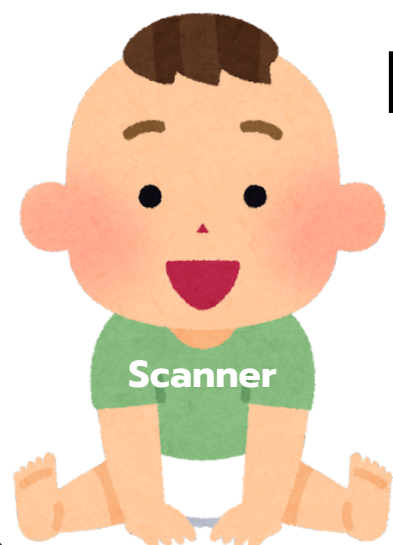


Tokens



¿Scanner = bebé?

Cuando un niño está aprendiendo un idioma, aunque aprende increíblemente rápido, es imposible que inmediatamente sepa todo el idioma. Entonces, se guía por las palabras que va reconociendo. Por ejemplo, en una oración escucha “mamá” o “papá” o “bebé”, y son palabras que va reconociendo y que le ayudan a separar los personajes. Esto mismo hace el scanner, busca las expresiones o palabras que conoce para poder separarla en fases.





4

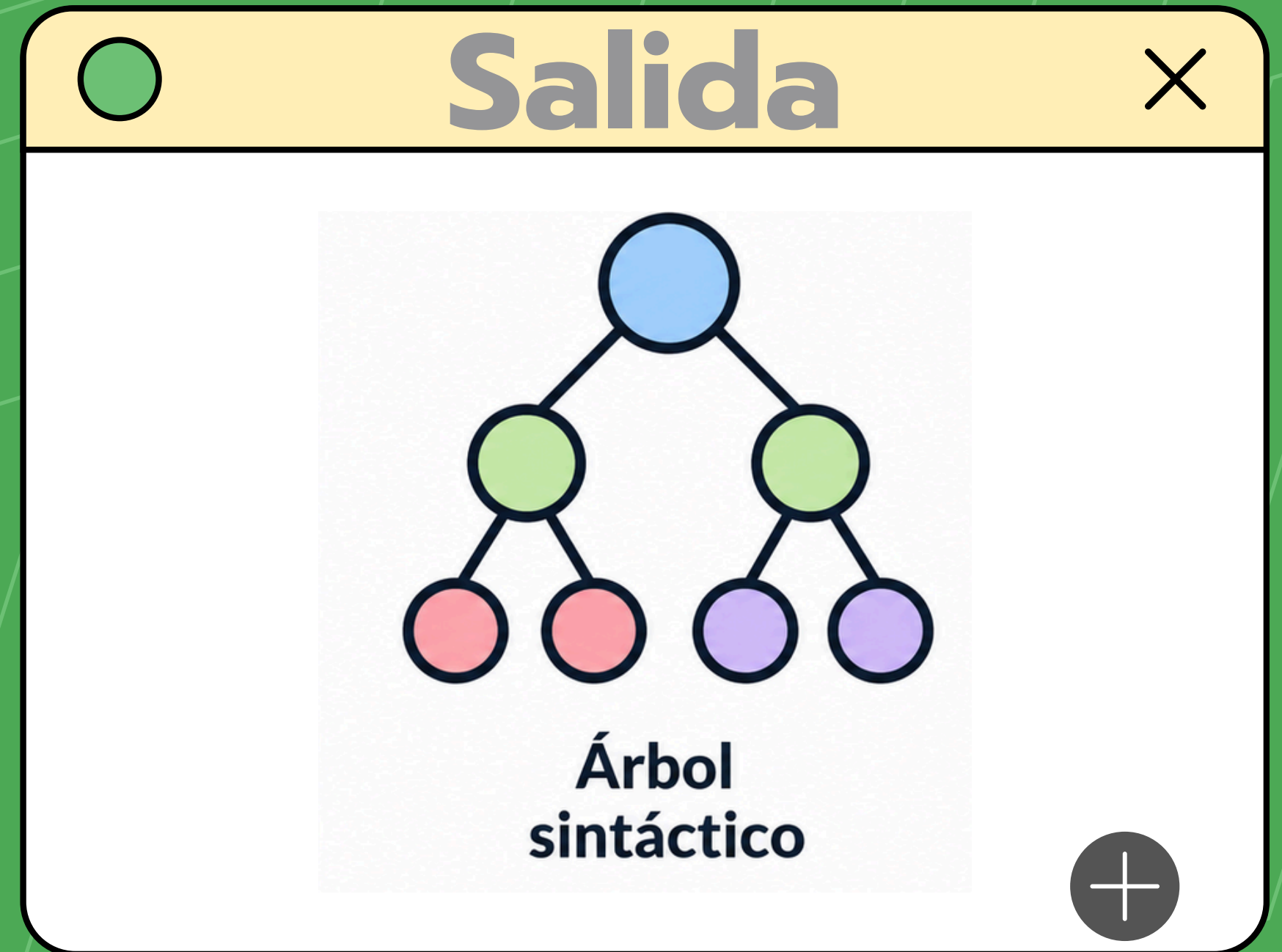
ANÁLISIS SINTÁCTICO (PARSER)

Análisis sintáctico

- El análisis sintáctico, conocido como parser, revisa que la sintaxis de un programa esté correcta.
- Toma la secuencia de tokens del scanner y verifica que correspondan a construcciones válidas.
- Genera un árbol sintáctico, poder armar un árbol significa que está sintácticamente correcto.
- Implementado con "Push-down Automata"
- De igual forma, hay herramientas que generan la mayor parte de un parser



Parser



5

ANÁLISIS SEMÁNTICO

Análisis semántico

- El analizador semántico se encarga de entender el significado del programa y revisar si tiene sentido lo que se pide que se haga.
- La semántica es estática, pues solo con ver el código fuente se puede deducir si hay cosas que no tienen sentido.
- El analizador sintáctico construye uno de los elementos más importantes, del que hemos hablado varias veces: la tabla de símbolos.
- Genera un árbol de artículos, al que llamaremos árbol decorado.

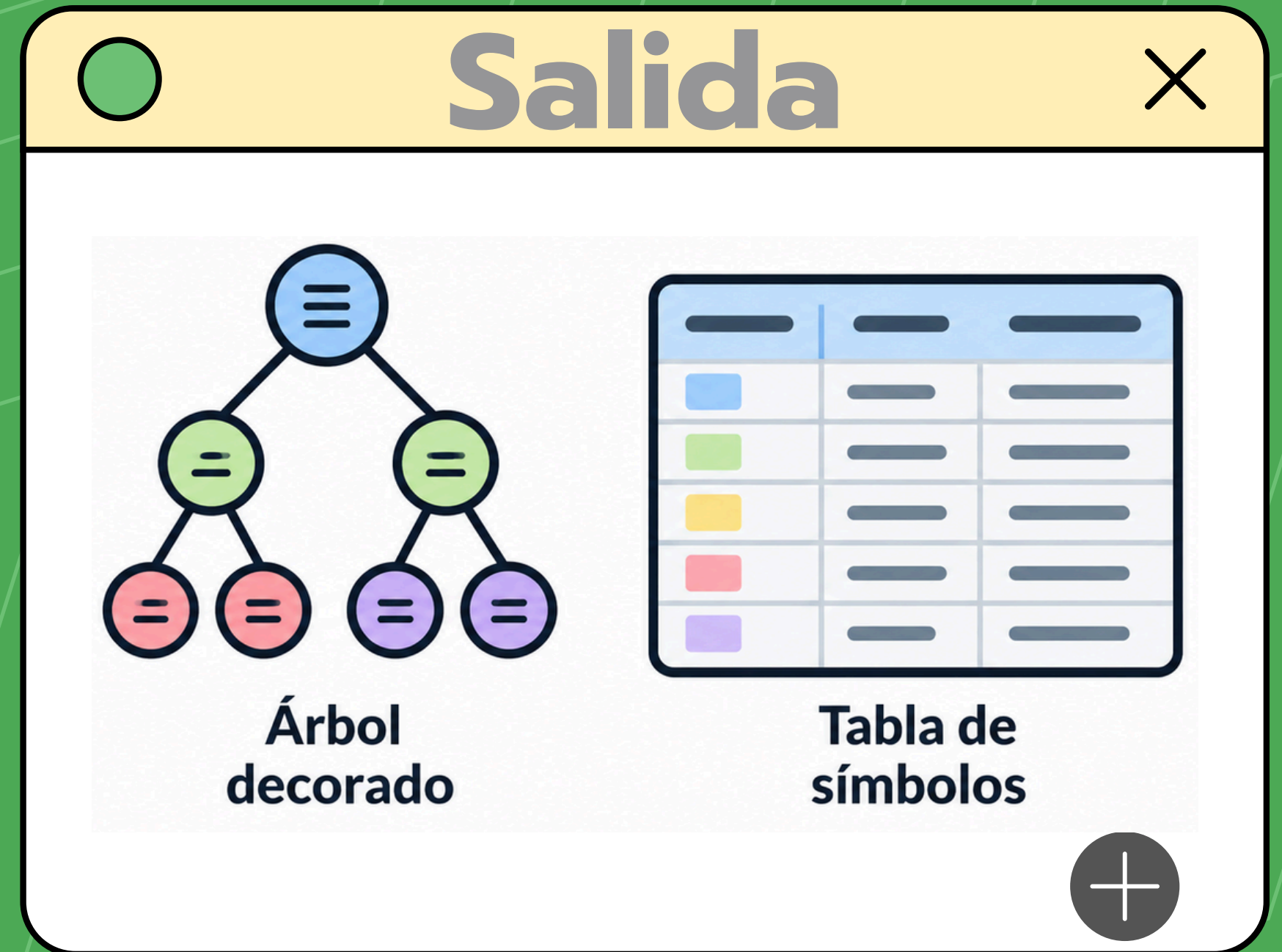
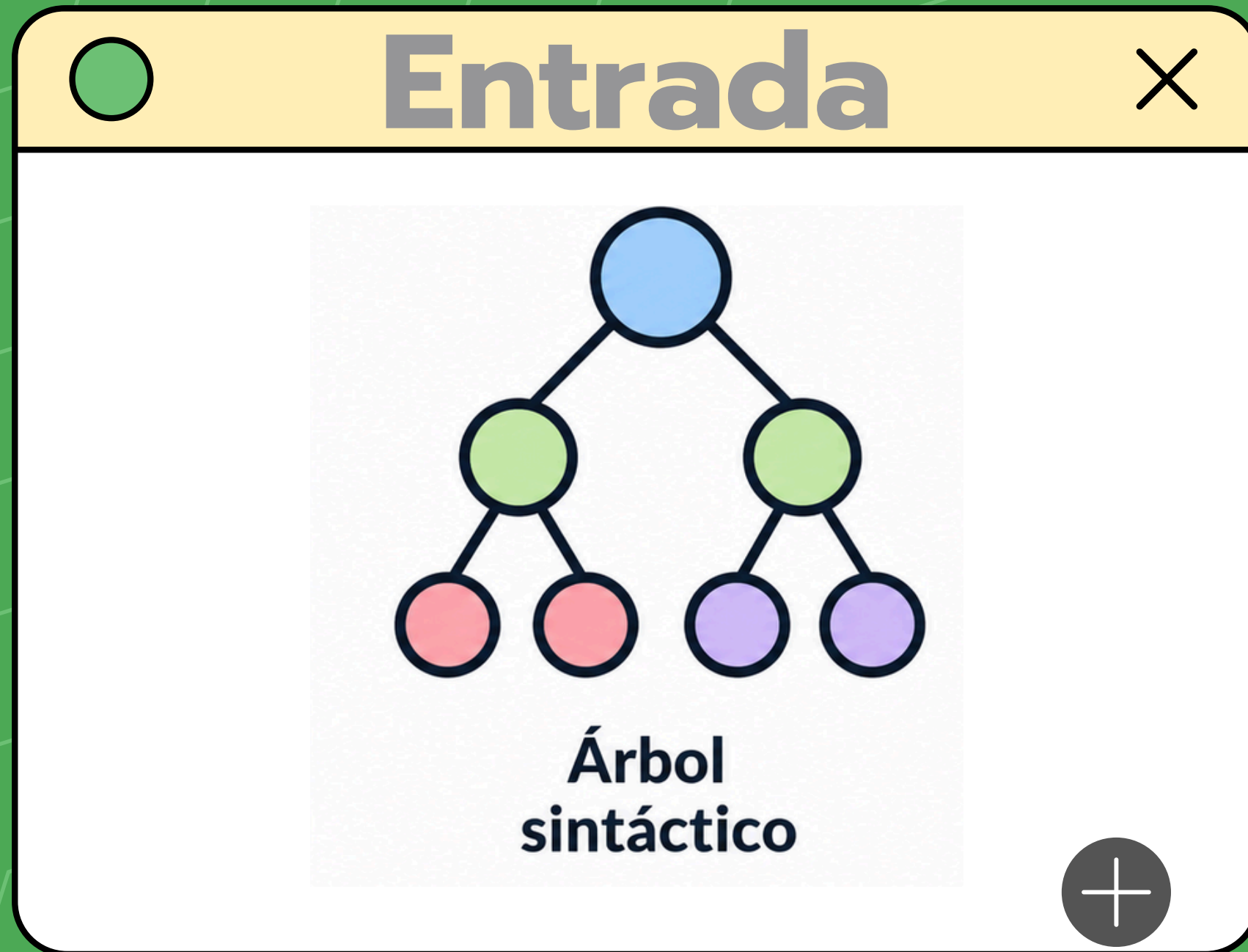


Error semántico vs sintáctico

Podemos mencionar algunos ejemplos de errores que son semánticos y no sintácticos. Por ejemplo, intentar obtener la raíz cuadrada de un string, no declarar el tipo de una variable si el lenguaje lo requiere, o, intentar sumar 2 tipos de datos no compatibles (un integer y un string por ejemplo). La semántica puede dar warnings cuando una parte del código tiene varios sentidos.



Analizador Semántico



6

OPTIMIZADOR DE FUENTE

Optimizador de fuente

- Introduce optimizaciones, se encarga de eliminar código innecesario antes de generar la representación intermedia.
- Realiza cálculo de constantes y propagación de constantes en sus lugares en el código.
- El paso de convertir el árbol a estructura lineal es sencillo, genera como salida la representación intermedia.
- Es similar a un ensamblador de 3 direcciones.

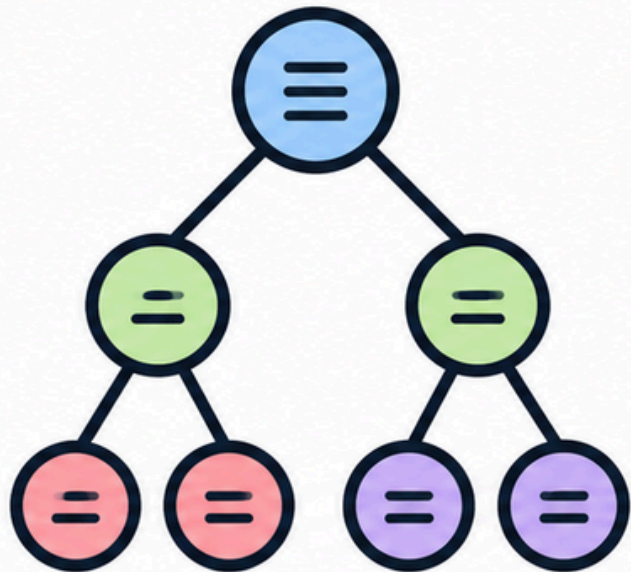


Optimizador de fuente

○

Entrada

×



Árbol decorado

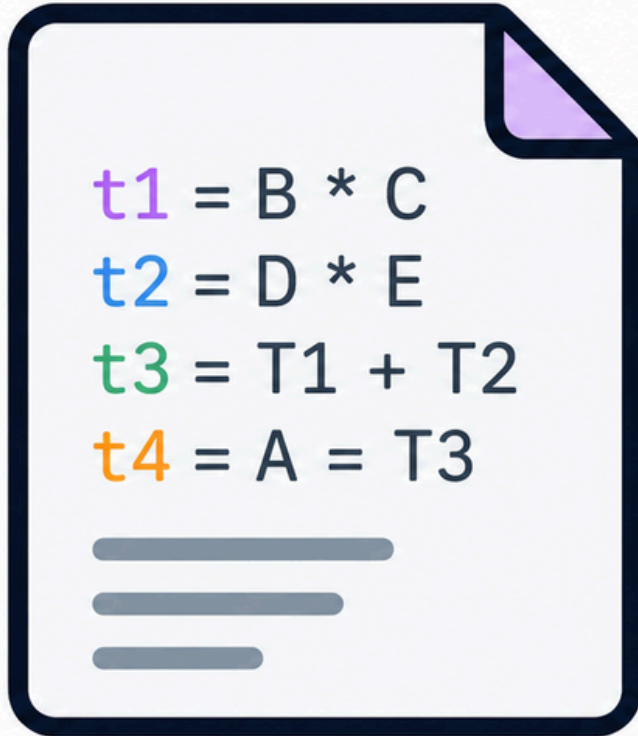
Tabla de símbolos

+

○

Salida

×



Representación intermedia

+

7

GENERADOR DE CÓDIGO

Generador de código

- Este empieza el backend del compilador, y se encarga de recibir el código intermedio y convertirlo en código real, puede ser ensamblador o lenguaje máquina dependiendo del compilador.
- Es importante la representación de datos, la cantidad de bytes y las restricciones de colocación.
- Hay que saber las posibilidades de la arquitectura para poder explotarlas.

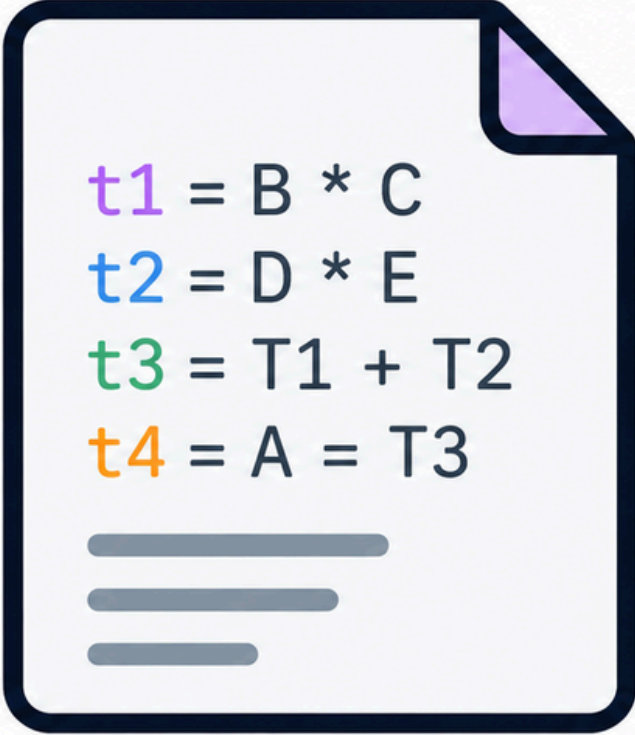


Generador de código

○

Entrada

×



```
t1 = B * C
t2 = D * E
t3 = T1 + T2
t4 = A = T3
```

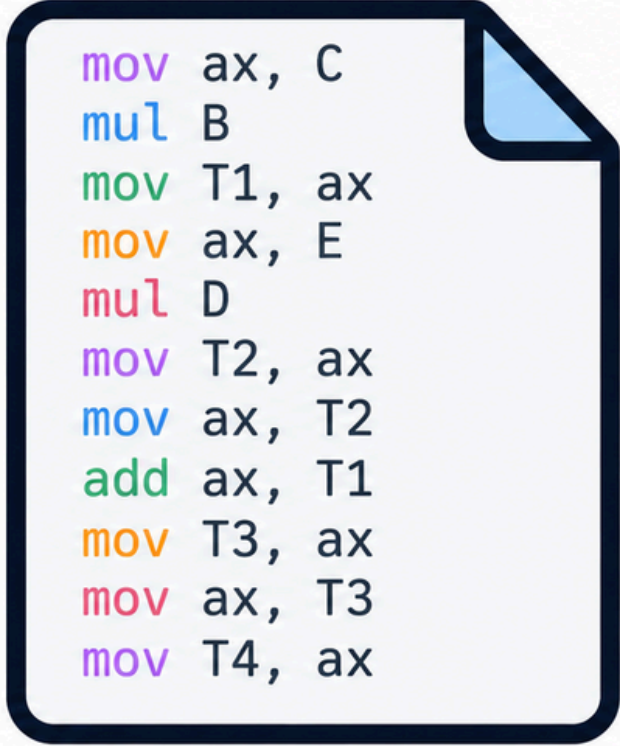
Representación
intermedia

+

○

Salida

×



```
mov ax, C
mul B
mov T1, ax
mov ax, E
mul D
mov T2, ax
mov ax, T2
add ax, T1
mov T3, ax
mov ax, T3
mov T4, ax
```

Lenguaje
máquina

+

Nótese que los datos son de un ejemplo real

8

OPTIMIZADOR DE CÓDIGO

Optimizador de código

- Se encarga de mejorar el código generado en el componente anterior, dando instrucciones que son equivalentes pero más rápidas.
- Da modos de direccionamiento más apropiados.
- Se encarga de eliminar código redundante y de eliminar código innecesario.



Optimizador de código

○

Entrada

×

```
mov ax, C
mul B
mov T1, ax
mov ax, E
mul D
mov T2, ax
mov ax, T2
add ax, T1
mov T3, ax
mov ax, T3
mov T4, ax
```

Lenguaje máquina

+

○

Salida

×

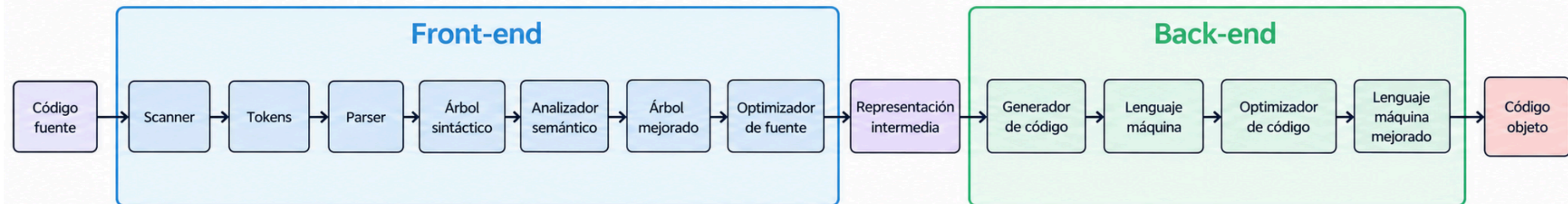
```
1 mov ax, C
2 mul B
3 mov dx, ax
4 mov ax, D
5 mul E
6 add ax, dx
7 mov A, ax
```

Lenguaje máquina mejorado

+

Nótese que los datos son de un ejemplo real

Fases de compilación (resumen)



Conclusión

Al fin pudimos ver la estructura básica de un compilador, pero, resulta que fuimos engañados...

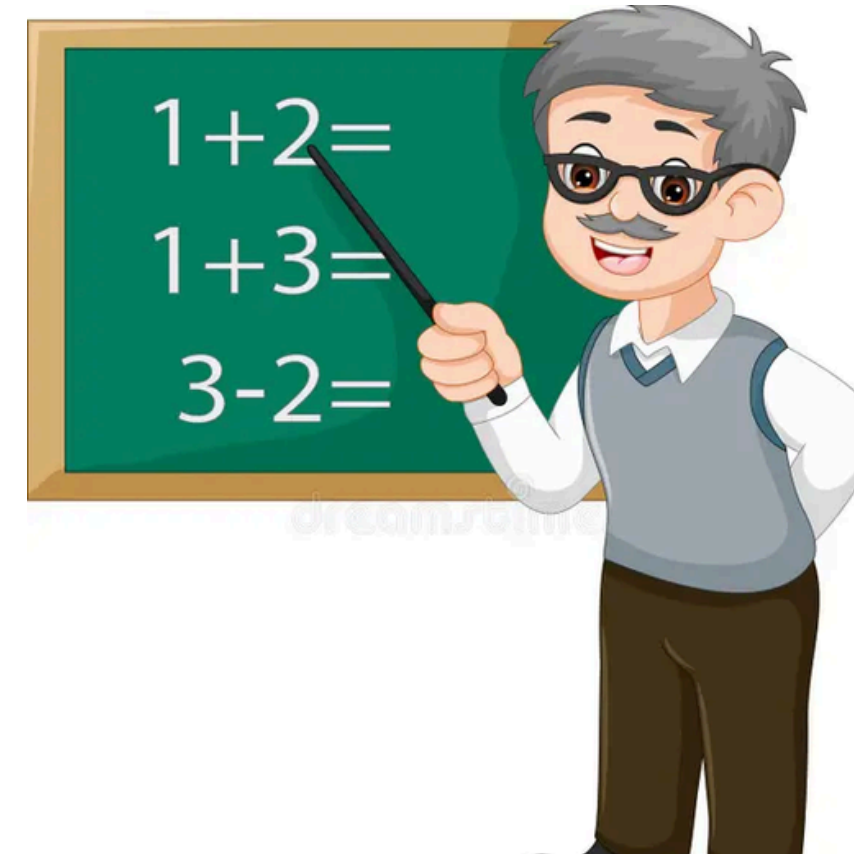
Es mentira que un compilador se vea así, pero se parece bastante. En los tiempos modernos se tienen compiladores en los que todos los componentes trabajan al mismo tiempo.



Encuentra las 7 diferencias



**Balón de oro
mintiéndole a Messi**



**Profesor de compi
mintiendo a sus alumnos**

**TO BE
CONTINUED...**

